Universidad de Guadalajara

Centro Universitario de Ciencias Exactas e Ingenierías

(CUCEI) Programación de sistemas avanzados

2026A

Carrera: Ingeniería en Computación

ENTREGA DE AVANCES

Equipo: Pyrrinoids

Repositorio: https://github.com/ZeusdG3/SistemasAvanzados

Integrantes:

VALERIO COMPARAN JUAN RAMON

GARCIA TOMAS CARLOS ANTONIO

CORONA PLASCENCIA ZEUS HADES

- Introducción

En el proyecto, se realizará el uso de una VPN mediante WireGuard. A su vez de la preparación de un entorno de contenedores que permita ejecutar, en una etapa posterior, un algoritmo computacional distribuido en el lenguaje de programación Rust.

Para ello es necesario utilizar un entorno virtual (VM), el cual correrá con sistema operativo Linux (Debian o Ubuntu). En este caso nosotros utilizaremos Ubuntu Server junto a HyperV para poder crear las máquinas virtuales correspondientes que se utilizaran para este proyecto.

Las restricciones dentro de este proyecto serán utilizar el sistema operativo sin entorno gráfico a través de la línea de comandos, no usar servicios de VPN administrados como ZeroTier o Tailscale. Así como servicios de nube como nodo central (hub) como AWS, Google Cloud y Azure. Como cualquier VPS de paga equivalente.

- Objetivo

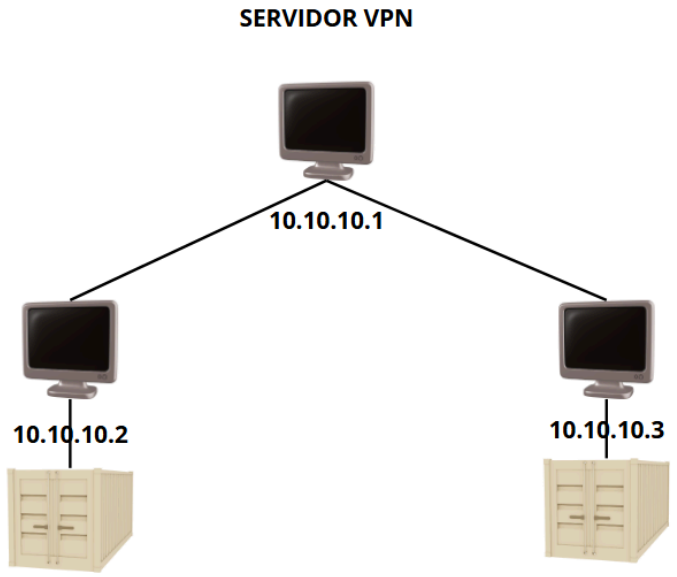
Diseñar, configurar y validar una red VPN privada entre los integrantes del equipo, así como preparar la infraestructura básica de contenedores Docker que se utilizará posteriormente para ejecutar un problema computacional de forma distribuida.

- Desarrollo (Avance)

Parte del Documento A

- Diagrama preliminar de la topología VPN.

Hub and spoke, centraliza las operaciones en el nodo principal (HUB) conectado a múltiples periféricos



- Descripción del rol de cada nodo.

RED VPN: 10.10.10.0/24

HUB: 10.10.10.1

Peer1: 10.10.10.2

Peer2: 10.10.10.3

Nodo 1 – HUB (10.10.10.1)

Rol: Servidor VPN

Actúa como servidor central de WireGuard

Administra las conexiones de los peers

Mantiene la tabla de peers autorizados

Permite la comunicación entre los nodos del cliente

Nodo 2 - Peer 1 (10.10.10.2)

Rol: Nodo Worker

Cliente de la VPN

Se conecta al HUB

Ejecuta contenedores Docker

Nodo 3 - Peer 2 (10.10.10.3)

Rol: Nodo Worker

Cliente de la VPN

Se conecta al HUB

Ejecuta contenedores Docker

- Evidencia de generación de llaves WireGuard.

Evidencia de generación de servidor

```
carlos@hub:~$ umask 077
carlos@hub:~$ wg genkey | tee server_private.key |
carlos@hub:~$ ls
server_private.key  server_public.key
carlos@hub:~$ server_private.key
server_private.key: command not found
carlos@hub:~$ cat sever_private.key
cat: sever_private.key: No such file or directory
carlos@hub:~$ ls
server_private.key  server_public.key
carlos@hub:~$ cat server_private.key
iO1oKo3vWHkiARdXF1rLEfV67xIqPOQQ2CXdcfIQNUs=
carlos@hub:~$ cat server_public.key
3ouLLxZsyxP5+Bxj8IsI6/vTpqF8YT91yEeNSPMmayQ=
carlos@hub:~$ _
```

Evidencia de generación de cliente

```
hades@hades:~$ ls
peer1_private.key  peer1_public.key
hades@hades:~$ cat peer1_private.key
gFeofn+ncBbfuRyidI6g2Dl/Dg7WIQbO+O8MVRuWS2k=
hades@hades:~$ cat peer1_public.key
j81jcGtlNTqffJHGE8mJ8JcdmkcqAqa1G5D9DMNq23k=
hades@hades:~$ _
```

- Evidencia de conectividad VPN entre al menos dos nodos.

Como nosotros aún no pudimos lograr la conexión exitosamente del VPN, pues claramente al ejecutar el programa nos saldra el error de que no hay conexión, como se muestra en la siguiente imagen:

```
juan@lacaprichosa:~/proyecto_sistemas_avanzados/rust$ cargo run
warning: struct `WorkerInfo` is never constructed
  --> src/messages.rs:16:12
   |
16 | pub struct WorkerInfo {
   |            ^^^^^^^^^^
   |
   |
   = note: `#[warn(dead_code)]` (part of `#[warn(unused)]`) on by default

warning: `algoritmo_distribuido` (bin "algoritmo_distribuido") generated 1 warning
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.15s
     Running `target/debug/algoritmo_distribuido`

thread 'main' (2850) panicked at src/main.rs:12:46:
called `Result::unwrap()` on an `Err` value: Os { code: 99, kind: AddrNotAvailable, message: "Cannot assign requested address" }
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace
juan@lacaprichosa:~/proyecto_sistemas_avanzados/rust$ ~
```

- Evidencia de instalación de Docker y Docker Compose.

docker-compose -f docker/docker-compose.yml build

```
juan@lacaprichosa:~/proyecto_sistemas_avanzados$ docker-compose -f docker/docker-compose.yml build
```

Después de ejecutar el comando y al finalizar el proceso nos saldrá lo siguiente:

```
Successfully built 538a776f85d1
Successfully tagged docker_worker4:latest
juan@lacaprichosa:~/proyecto_sistemas_avanzados$
```

Para levantar los contenedores que acabamos de crear es con el siguiente comando:

docker-compose -f docker/docker-compose.yml up -d

Nos saldrá lo siguiente:

```
juan@lacaprichosa:~/proyecto_sistemas_avanzados$ docker-compose -f docker/docker-compose.yml up -d
Creating docker_worker2_1     ... done
Creating docker_coordinator_1 ... done
Creating docker_worker4_1     ... done
Creating docker_worker3_1     ... done
Creating docker_worker1_1     ... done
juan@lacaprichosa:~/proyecto_sistemas_avanzados$ _
```

Y con esto tendremos los contenedores levantados en nuestra máquina, cabe mencionar que como la VPN no está en funcionamiento, por como esta configurado el codigo en Rust los contenedores se detendrán al momento, ya que falta hacerlos tolerantes a fallos.
Pero podemos ver que si se ejecutaron y su tiempo vivo:

```
juan@lacaprichosa:~/proyecto_sistemas_avanzados$ docker ps -a
CONTAINER ID   IMAGE                COMMAND                  CREATED        STATUS                       PORTS     NAMES
01abb3149a37   docker_worker1       "./algoritmo_distrib…"   26 hours ago   Up 30 seconds                          docker_worke
r1_1
c64249c752d3   docker_worker3       "./algoritmo_distrib…"   26 hours ago   Up 30 seconds                          docker_worke
r3_1
ec816d2b3a5f   docker_coordinator   "./algoritmo_distrib…"   26 hours ago   Exited (101) 30 seconds ago            docker_coord
inator_1
a1f677fdb9e7   docker_worker2       "./algoritmo_distrib…"   26 hours ago   Up 30 seconds                          docker_worke
r2_1
4df7db701087   docker_worker4       "./algoritmo_distrib…"   26 hours ago   Up 30 seconds                          docker_worke
r4_1
```

Y como mencionaba anteriormente, al ejecutarse el codigo da un error por la configuración que hay en el algoritmo que pusimos de momento, nos da el siguiente log:

```
juan@lacaprichosa:~/proyecto_sistemas_avanzados$ docker logs docker_worker1_1

thread 'main' (1) panicked at src/main.rs:10:36:
called `Result::unwrap()` on an `Err` value: Os { code: 101, kind: NetworkUnreachable, message: "Network is unreachable" }
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace

thread 'main' (1) panicked at src/main.rs:10:36:
called `Result::unwrap()` on an `Err` value: Os { code: 101, kind: NetworkUnreachable, message: "Network is unreachable" }
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace
```

- Captura del docker-compose.yml base.

```yaml
version: "3.8"
services:
  coordinator:
    build:
      context: ..
      dockerfile: docker/Dockerfile
    command: ./algoritmo_distribuido
    network_mode: "host"

  worker1:
    build:
      context: ..
      dockerfile: docker/Dockerfile
    command: ./algoritmo_distribuido worker
    network_mode: "host"

  worker2:
    build:
      context: ..
      dockerfile: docker/Dockerfile
    command: ./algoritmo_distribuido worker
    network_mode: "host"

  worker3:
    build:
      context: ..
      dockerfile: docker/Dockerfile
    command: ./algoritmo_distribuido worker
    network_mode: "host"

  worker4:
    build:
      context: ..
      dockerfile: docker/Dockerfile
    command: ./algoritmo_distribuido worker
    network_mode: "host"
```

- Descripción de problemas encontrados y decisiones técnicas tomadas.

Curva de aprendizaje en linux sin entorno gráfico.

Hemos tenido que visualizar los comandos que son necesarios para poder realizar la práctica de manera correspondiente, es muy importante verificar las configuraciones ya que al no contar con entorno gráfico cualquier cambio aplicado solamente se puede acceder mediante comandos.

Configuración de Wireguard.

Nosotros hemos encontrado incidencias a la hora de hacer ping entre nodos para poder tener conexión dentro de la VPN, hemos configurado correctamente los archivos wg0 de manera correspondiente, teniendo en cuenta el Port Forwarding del modem, así como no contar con CGNAT y contar con una IP pública que permita poder ser el HUB.

Docker.

Para poder levantar los docker necesitamos conexión de manera que se pueda verificar, por lo que al no poder tener conexión entre nodos no fue posible mantener una conexión entre contenedores entre máquinas virtuales.

- Definición y justificación del problema computacional distribuido a implementar.

Se decidió utilizar el modelo hub and spoke debido a que simplifica la configuración inicial, reduce los errores de enrutamiento y centraliza la administración.

Se decidieron usar IPs estáticas debido a que simplifica la configuración del algoritmo

Debido al tamaño del equipo se optó por usar Docker Compose.

Parte del Documento B

A continuación se muestran avances de los archivos creados para hacer funcionar el algoritmo distribuido en Rust, hasta el momento, cuando se ejecuta como "Coordinador" este mismo esperada a que los "Workers" se conecten para que este pueda repartir el algoritmo en cuestión, una vez que los Workers terminan su parte, la regresan al Coordinador.

Como aun no esta implementado el algoritmo a usar, se envía un mensaje al Worker donde dice "Resultado "dummy" " donde se debería ver el resultado parcial del worker, mientras que tanto en el Coordinador como Worker cuando la conexión fue exitosamente, saldra un mensaje diciendo "Worker conectado".

Messages.rs

```rust
use serde::{Serialize, Deserialize};

#[derive(Serialize, Deserialize, Debug)]
pub struct Task {
    pub id: u32,
    pub payload: String, // Aqui es donde iria la tarea real
}

#[derive(Serialize, Deserialize, Debug)]
pub struct ResultMsg {
    pub id: u32,
    pub output: String, // Aqui va el resultado parcial
}

#[derive(Serialize, Deserialize, Debug)]
pub struct WorkerInfo {
    pub id: u32,
    pub address: String,
}

```

coordinator.rs

```rust
use crate::messages::{Task, ResultMsg};
use tokio::net::TcpListener;
use tokio::io::{AsyncWriteExt, AsyncReadExt};

pub async fn run_coordinator() -> tokio::io::Result<()> {
    let listener = TcpListener::bind("10.10.10.1:8080").await?;
    println!("Coordinador escuchando en 10.10.10.1:8080");

    loop {
        let (mut socket, _) = listener.accept().await?;
        println!("Worker conectado");

        //Aqui envia la tarea dummy
        let task = Task { id: 1, payload: "dummy".to_string() };
        let serialized = serde_json::to_string(&task).unwrap();
        socket.write_all(serialized.as_bytes()).await?;

        //Aqui recibe el resultado
        let mut buf = vec![0; 1024];
        let n = socket.read(&mut buf).await?;
        let response: ResultMsg = serde_json::from_slice(&buf[..n]).unwrap();
        println!("Resultado recibido: {:?}", response);

    }
}

```

worker.rs

[coordinator.rs] micro worker.rs

```rust
use crate::messages::{Task, ResultMsg};
use tokio::net::TcpListener;
use tokio::io::{AsyncWriteExt, AsyncReadExt};

pub async fn run_coordinator() -> tokio::io::Result<()> {
    let listener = TcpListener::bind("10.10.10.1:8080").await?;
    println!("Coordinador escuchando en 10.10.10.1:8080");

    loop {
        let (mut socket, _) = listener.accept().await?;
        println!("Worker conectado");

        //Aqui envia la tarea dummy
        let task = Task { id: 1, payload: "dummy".to_string() };
        let serialized = serde_json::to_string(&task).unwrap();
        socket.write_all(serialized.as_bytes()).await?;

        //Aqui recibe el resultado
        let mut buf = vec![0; 1024];
        let n = socket.read(&mut buf).await?;
        let response: ResultMsg = serde_json::from_slice(&buf[..n]).unwrap();
        println!("Resultado recibido: {:?}", response);

    }
}

```

main.rs

```rust
mod coordinator;
mod worker;
mod messages;

#[tokio::main]
async fn main() {
    let args: Vec<String> = std::env::args().collect();

    if args.len() > 1 && args[1] == "worker" {
        worker::run_worker().await.unwrap();
    } else {
        coordinator::run_coordinator().await.unwrap();
    }
}

```

Como nosotros aún no pudimos lograr la conexión exitosamente del VPN, pues claramente al ejecutar el programa nos saldra el error de que no hay conexión, como se muestra en la siguiente imagen:

```
juan@lacaprichosa:~/proyecto_sistemas_avanzados/rust$ cargo run
warning: struct `WorkerInfo` is never constructed
  --> src/messages.rs:16:12
   |
16 | pub struct WorkerInfo {
   |            ^^^^^^^^^^
   |
   = note: `#[warn(dead_code)]` (part of `#[warn(unused)]`) on by default

warning: `algoritmo_distribuido` (bin "algoritmo_distribuido") generated 1 warning
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.15s
     Running `target/debug/algoritmo_distribuido`

thread 'main' (2850) panicked at src/main.rs:12:46:
called `Result::unwrap()` on an `Err` value: Os { code: 99, kind: AddrNotAvailable, message: "Cannot assign requested address" }
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace
juan@lacaprichosa:~/proyecto_sistemas_avanzados/rust$ ~
```

Creacion y construccion de imagen y contenedores de docker

Primero definimos el Dockerfile y el docker-compose en el directorio de docker de nuestro proyecto.

Dockerfile

```
# Imagen base con Rust
FROM rust:latest AS builder

WORKDIR /usr/src/app

# Copiar proyecto Rust desde /rust

COPY rust .

# Compilar en modo release

RUN cargo build --release



FROM rust:latest

WORKDIR /usr/src/app

# Copiar solo el binario compilado

COPY --from=builder /usr/src/app/target/release/algoritmo_distribuido .

# Comando por defecto

CMD ["./algoritmo_distribuido"]

```

docker-compose.yml

```yaml
version: "3.8"
services:
  coordinator:
    build:
      context: ..
      dockerfile: docker/Dockerfile
    command: ./algoritmo_distribuido
    network_mode: "host"

  worker1:
    build:
      context: ..
      dockerfile: docker/Dockerfile
    command: ./algoritmo_distribuido worker
    network_mode: "host"

  worker2:
    build:
      context: ..
      dockerfile: docker/Dockerfile
    command: ./algoritmo_distribuido worker
    network_mode: "host"

  worker3:
    build:
      context: ..
      dockerfile: docker/Dockerfile
    command: ./algoritmo_distribuido worker
    network_mode: "host"

  worker4:
    build:
      context: ..
      dockerfile: docker/Dockerfile
    command: ./algoritmo_distribuido worker
    network_mode: "host"

```

Después de definir los valores de configuración, pasamos a crear la imagen del algoritmo con el siguiente comando:

docker-compose -f docker/docker-compose.yml build

Como se muestra en la siguiente imagen:

```
juan@lacaprichosa:~/proyecto_sistemas_avanzados$ docker-compose -f docker/docker-compose.yml build
```

Después de ejecutar el comando y al finalizar el proceso nos saldrá lo siguiente:

```
Successfully built 538a776f85d1
Successfully tagged docker_worker4:latest
juan@lacaprichosa:~/proyecto_sistemas_avanzados$
```

Para levantar los contenedores que acabamos de crear es con el siguiente comando:

docker-compose -f docker/docker-compose.yml up -d

Nos saldrá lo siguiente:

```
juan@lacaprichosa:~/proyecto_sistemas_avanzados$ docker-compose -f docker/docker-compose.yml up -d
Creating docker_worker2_1     ... done
Creating docker_coordinator_1 ... done
Creating docker_worker4_1     ... done
Creating docker_worker3_1     ... done
Creating docker_worker1_1     ... done
juan@lacaprichosa:~/proyecto_sistemas_avanzados$ _
```

Y con esto tendremos los contenedores levantados en nuestra máquina.

Plan de ejecución (en bullets):

Qué falta implementar, qué probarán primero, cómo medirán rendimiento, y cómo harán la prueba de falla.

Para la medición de rendimiento en el sistema distribuido se usarán las siguientes herramientas:

- **Tiempo de ejecución total:** medición con `**time**` en Linux o `**Instant**` en Rust.
- **Latencia de comunicación:** timestamps en mensajes y pruebas de red con `**ping**` o `**iperf3**`.
- **Throughput:** contador de tareas procesadas por unidad de tiempo en el coordinador.
- **Uso de recursos:** monitoreo con `**docker stats**`, `**htop**` o `**top**`.
- **Escalabilidad:** ejecución con distinto número de workers y comparación de resultados.
- **Tolerancia a fallos:** pruebas deteniendo contenedores para observar la reacción del coordinador.

Parte del Documento C

Metodología base

Comunicación activa mediante WhatsApp

Se utilizará un tipo de Scrum ligero

Sprints cortos de una semana

Reuniones mediante Google Meet

Uso de trello como herramienta de gestión (Product Backlog y Sprint)

Roles

Los 3 integrantes participan en:

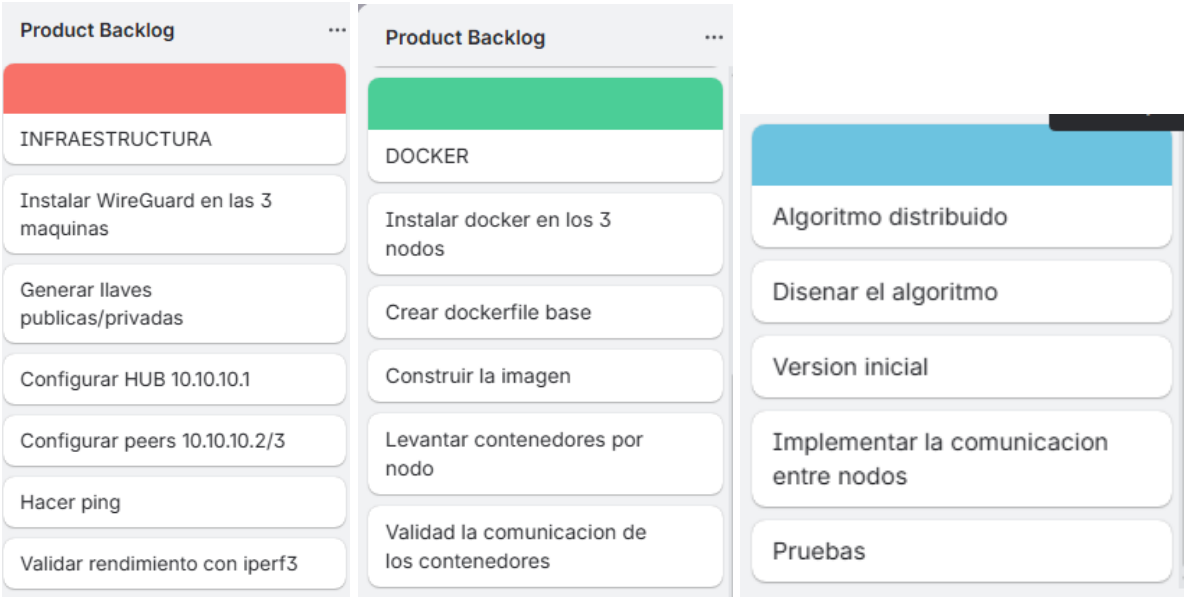
- Configuración de infraestructura
- Desarrollo del algoritmo
- Documentación y pruebas

Responsable técnico (Rotativo por sprint)

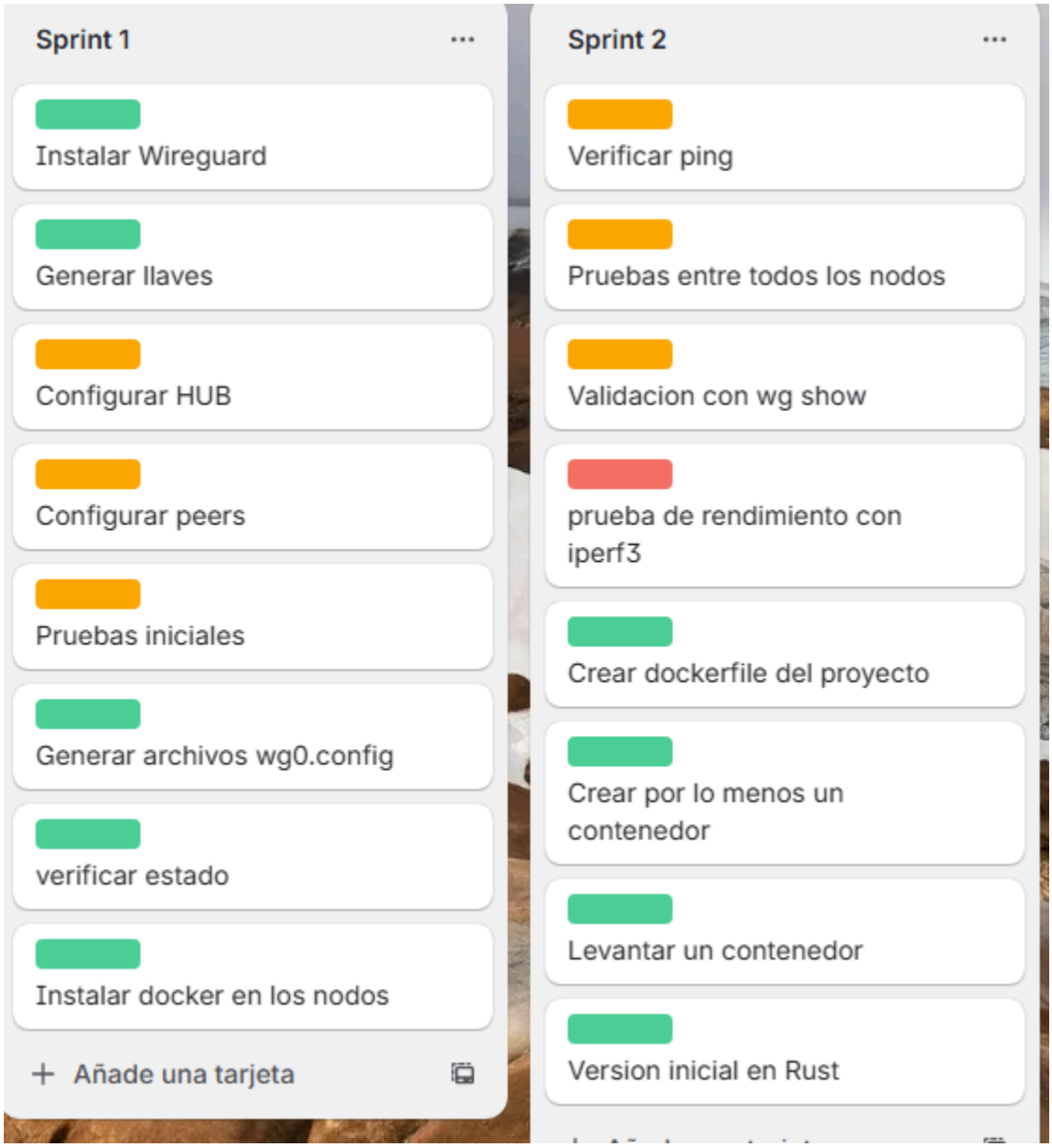
- Supervisar la configuración
- Coordinar las tareas del sprint
- Validar las pruebas

Artefactos (Evidencia presentable)

Product Backlog



Sprints


Sprint 1
Instalar Wireguard
Generar llaves
Configurar HUB
Configurar peers
Pruebas iniciales
Generar archivos wg0.config
verificar estado
Instalar docker en los nodos
Añade una tarjeta
Sprint 2
Verificar ping
Pruebas entre todos los nodos
Validacion con wg show
prueba de rendimiento con iperf3
Crear dockerfile del proyecto
Crear por lo menos un contenedor
Levantar un contenedor
Version inicial en Rust

Pruebas de minutas

A	B	C	D	E	F
Nombre	**Apellido**	**Correo electrónico**	**Duración**	**Hora a la que se unió**	**Hora a la que salió**
CARLOS ANTONIO	GARCIA TOMAS	carlos.garcia6654@alumnos.udg.mx	2 h 30 min	20:13	22:43
JUAN RAMON	VALERIO COMPARAN	juan.valerio5617@alumnos.udg.mx	2 h 31 min	20:12	22:43
ZEUS HADES	CORONA PLASCENCIA	zeus.corona0285@alumnos.udg.mx	2 h 30 min	20:13	22:43

A	B	C	D	E	F
Nombre	**Apellido**	**Correo electrónico**	**Duración**	**Hora a la que se unió**	**Hora a la que salió**
Hades	Corona	copl*********@***.com	2 h 31 min	15:58	19:02
ZEUS HADES	CORONA PLASCENCIA	zeus.corona0285@alumnos.udg.mx	2 h 52 min	15:47	19:02
CARLOS ANTONIO	GARCIA TOMAS	carlos.garcia6654@alumnos.udg.mx	3 h 19 min	15:43	19:02
JUAN RAMON	VALERIO COMPARAN	juan.valerio5617@alumnos.udg.mx	3 h 19 min	15:42	19:02

Repositorio Github

https://github.com/ZeusdG3/SistemasAvanzados

Conclusión (Avance)

En este punto hemos entendido varios puntos los cuales son vitales para que la práctica funciones por ende, las configuraciones correspondientes de la VPN son vitales para que la red de contenedores funcione y puedan hacer de manera correcta programación paralela.

WSL2 era nuestra configuración inicial la cual presentó problemas al realizarla como HUB dentro de la configuración de Windows por el firewall y porque mantiene una IP diferente y cambiante cada que se reinicia el equipo, sin embargo, puede utilizarse como cliente ya que no es necesario mantener un enlace fijo.

Referencias

De Luz, S. (2020, enero 13). WireGuard VPN: Instalación y configuración de la mejor VPN. Recuperado el 20 de febrero de 2026, de RedesZone website: https://www.redeszone.net/tutoriales/vpn/wireguard-vpn-configuracion/

Instalar Hyper-V. (s/f). Recuperado el 20 de febrero de 2026, de Microsoft.com website: https://learn.microsoft.com/es-es/windows-server/virtualization/hyper-v/get-started/install-hyper-v?tabs=powershell&pivots=windows

Donenfeld, J. A. (s/f). WireGuard: fast, modern, secure VPN tunnel. Recuperado el 20 de febrero de 2026, de Wireguard.com website: https://www.wireguard.com/

Get Ubuntu Server. (s/f). Recuperado el 20 de febrero de 2026, de Ubuntu website: https://ubuntu.com/download/server